

Top 20 JavaScript Questions for Senior Developers (2026)

Curated interview questions for developers with 5+ years of experience — engine internals, system design, advanced async, architecture, security at scale, and the kind of trade-off discussions expected in senior and staff-level loops.

Table of Contents

1. [Implement a Promise that conforms to the Promises/A+ spec](#)
2. [How does JavaScript garbage collection work?](#)
3. [What is the difference between the browser and Node.js event loops?](#)
4. [Implement `Promise.all` and explain failure semantics](#)
5. [How do Web Workers, Worker Threads, and SharedArrayBuffer work?](#)
6. [Design and implement an EventEmitter / pub-sub system](#)
7. [Explain property descriptors, Symbols, and metaprogramming](#)
8. [How does V8 optimize JavaScript execution?](#)

9. [What are backpressure and streaming patterns in Node.js?](#)
 10. [Implement function composition — `compose` and `pipe`](#)
 11. [How do you architect JavaScript for microfrontends?](#)
 12. [What are SSR and hydration pitfalls in isomorphic JavaScript?](#)
 13. [How do Service Workers enable offline-first applications?](#)
 14. [How do you structure and scale a large JavaScript codebase?](#)
 15. [What is JavaScript supply chain security and how do you harden it?](#)
 16. [Implement deep equality with edge case handling](#)
 17. [Compare Promises, Observables, and async iterators](#)
 18. [How do you diagnose performance in long-running SPAs?](#)
 19. [When and how do you integrate WebAssembly with JavaScript?](#)
 20. [System design: build a real-time notification system in JavaScript](#)
-

1. Implement a Promise that conforms to the Promises/A+ spec

Answer: Senior candidates should implement (or whiteboard) a minimal Promise covering: executor, `then` chaining, value

assimilation, and async resolution.

```

const PENDING = "pending";
const FULFILLED = "fulfilled";
const REJECTED = "rejected";

class MyPromise {
  #state = PENDING;
  #value = undefined;
  #handlers = [];

  constructor(executor) {
    const resolve = (value) => this.#settle(FULFILLED, value);
    const reject = (reason) => this.#settle(REJECTED, reason);

    try {
      executor(resolve, reject);
    } catch (err) {
      reject(err);
    }
  }

  #settle(state, value) {
    if (this.#state !== PENDING) return;

    // Assimilate thenables (Promises/A+ 2.3)
    if (value && typeof value.then === "function") {
      value.then(
        (v) => this.#settle(FULFILLED, v),

```

```

        (r) => this.#settle(REJECTED, r)
    );
    return;
}

this.#state = state;
this.#value = value;
this.#handlers.forEach((h) => this.#runHandler(h));
this.#handlers = null;
}

#runHandler({ onFulfilled, onRejected, resolve,
reject }) {
    queueMicrotask(() => {
        try {
            if (this.#state === FULFILLED) {
                const result = onFulfilled ?
onFulfilled(this.#value) : this.#value;
                resolve(result);
            } else {
                const result = onRejected ?
onRejected(this.#value) : this.#throw(this.#value);
                resolve(result);
            }
        } catch (err) {
            reject(err);
        }
    });
}

#throw(reason) {
    throw reason;
}

```

```

}

then(onFulfilled, onRejected) {
  return new MyPromise((resolve, reject) => {
    const handler = { onFulfilled, onRejected,
resolve, reject };

    if (this.#state === PENDING) {
      this.#handlers.push(handler);
    } else {
      this.#runHandler(handler);
    }
  });
}

catch(onRejected) {
  return this.then(null, onRejected);
}

finally(onFinally) {
  return this.then(
    (v) => MyPromise.resolve(onFinally?.()).then(()
=> v),
    (r) => MyPromise.resolve(onFinally?.()).then(()
=> { throw r; })
  );
}

static resolve(value) {
  return new MyPromise((res) => res(value));
}

```

```
static reject(reason) {
  return new MyPromise((_, rej) => rej(reason));
}
}
```

Key spec rules to mention:

- `then` must return a new Promise
- `onFulfilled` / `onRejected` are optional — values pass through
- Handlers run asynchronously (microtask), never synchronously
- Thenable assimilation prevents infinite loops with identity checks

Interview extension: Explain why `new Promise(r => r(Promise.resolve(1)))` does not double-wrap — assimilation unwraps nested thenables.

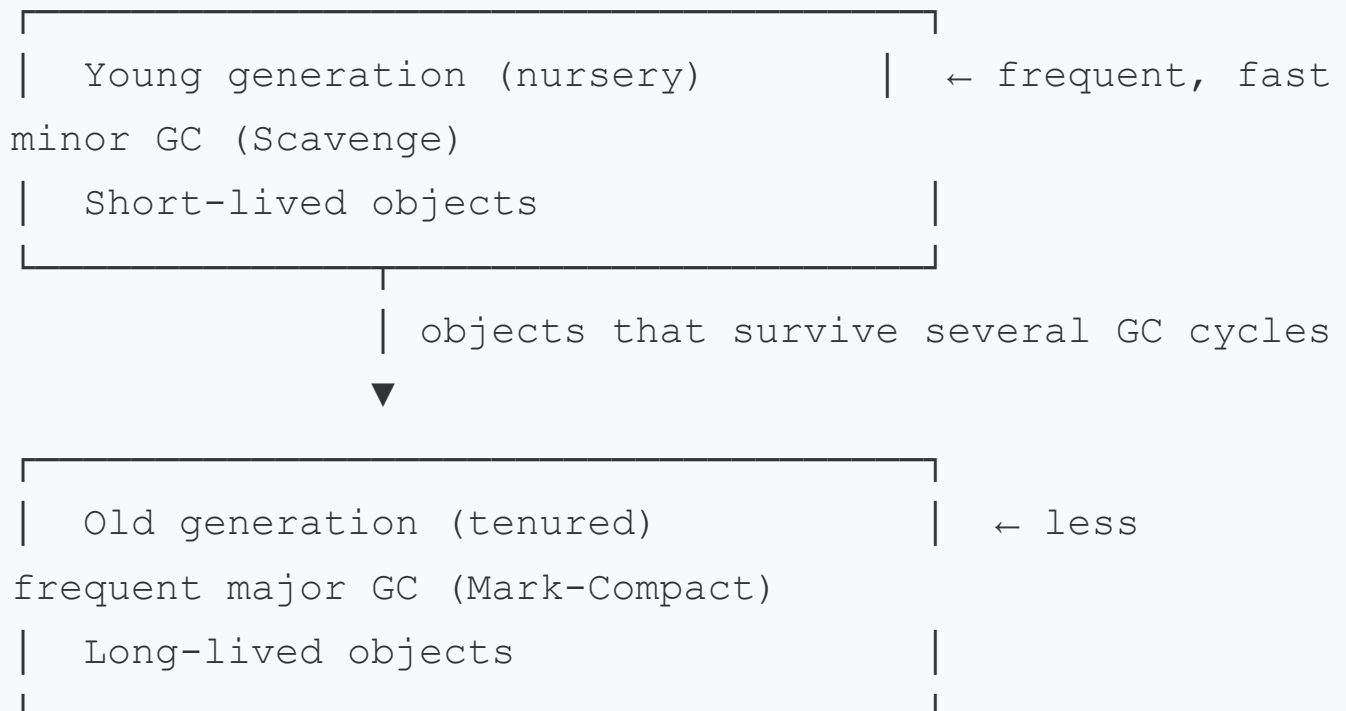
2. How does JavaScript garbage collection work?

Answer: JavaScript uses **automatic garbage collection**. Modern engines (V8, SpiderMonkey, JavaScriptCore) primarily use **generational mark-and-sweep** with incremental and concurrent collection to reduce pause times.

Mark-and-sweep (conceptual):

1. Start from **GC roots** (global object, call stack locals, closures, DOM references in browser)
2. **Mark** all reachable objects by traversing references
3. **Sweep** unmarked objects and reclaim memory

Generational hypothesis: Most objects die young.



V8 optimizations (2026):

- **Incremental marking** — split marking work across event loop turns
- **Concurrent marking/sweeping** — background threads reduce main-thread pauses
- **Orinoco (V8)** — parallel and concurrent GC architecture

- **Write barriers** — track when old-gen objects reference young-gen objects

What keeps objects alive:

- Global variables
- Closures referencing outer scope
- DOM nodes referenced from JS (browser)
- Active timers, event listeners, open sockets (Node.js)
- `FinalizationRegistry` callbacks (weak — does not prevent collection)

Senior talking points:

- GC is non-deterministic — do not rely on `delete` or `null` assignment for immediate cleanup
 - Large `ArrayBuffer` / `TypedArray` may live outside standard heap (V8 external memory)
 - `--max-old-space-size` in Node.js for memory tuning under load
 - Profile with heap snapshots — look for **retainers** and **detached DOM trees**
-

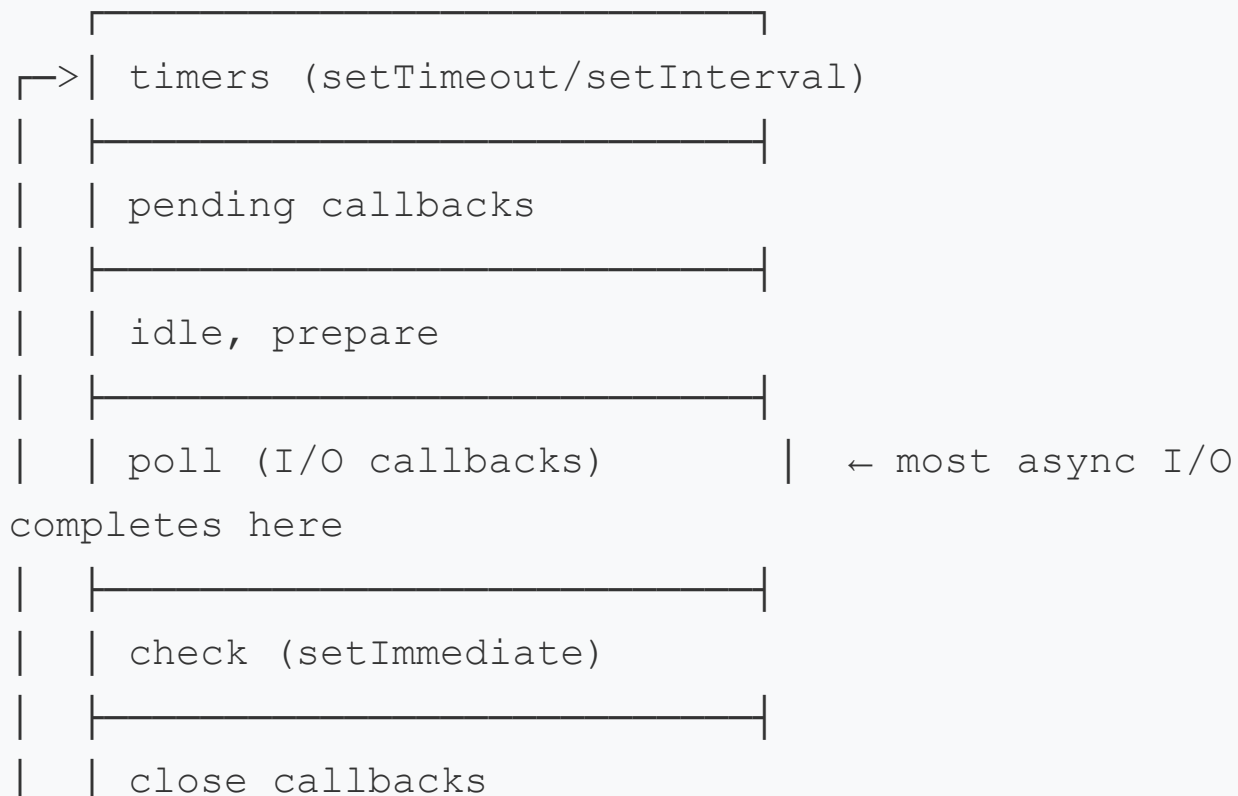
3. What is the difference between the browser and Node.js event loops?

Answer: Both use a single-threaded event loop model, but the **phase structure** and **I/O integration** differ.

Browser event loop (simplified):

1. Run macrotask (one)
2. Drain microtask queue (all)
3. Render (if needed) – style, layout, paint
4. Repeat

Node.js event loop (libuv phases):



↓ after each phase: drain microtasks

Key differences:

Aspect	Browser	Node.js
Rendering	Integrated (rAF)	N/A
I/O	Web APIs	libuv thread pool + OS async I/O
<code>setImmediate</code>	Not available	Runs in check phase (after poll)
<code>setTimeout(0)</code> vs <code>setImmediate</code>	N/A	Order varies by context
Microtasks	Same turn after macrotask	Same — <code>process.nextTick</code> runs before microtasks

```
// Node.js ordering trap
setTimeout(() => console.log("timeout"), 0);
setImmediate(() => console.log("immediate"));
process.nextTick(() => console.log("nextTick"));
```

```
Promise.resolve().then(() => console.log("promise"));

// Typical output:
// nextTick → promise → timeout/immediate (order of
// last two depends on context)
```

Senior insight: `process.nextTick` can starve I/O if used recursively — prefer `queueMicrotask` or `setImmediate` for deferral in Node.js server code.

4. Implement `Promise.all` and explain failure semantics

Answer:

```
function promiseAll(iterable) {
  return new Promise((resolve, reject) => {
    const items = Array.from(iterable);
    if (items.length === 0) return resolve([]);

    const results = new Array(items.length);
    let remaining = items.length;

    items.forEach((item, index) => {
      Promise.resolve(item)
        .then((value) => {
          results[index] = value;
          if (--remaining === 0) resolve(results);
        })
        .catch(reject);
    });
  });
}
```

```

    })
    .catch(reject); // fail-fast on first rejection
  });
});
}

```

Semantics senior devs must articulate:

- Input can contain non-Promise values — wrapped via `Promise.resolve`
- **Order preserved** — result index matches input index regardless of completion order
- **Fail-fast** — first rejection rejects the entire `Promise.all`; other promises keep running but results are discarded
- Empty iterable → immediately resolves `[]`
- For partial success use `Promise.allSettled`

Related implementation — `Promise.race`:

```

function promiseRace(iterable) {
  return new Promise((resolve, reject) => {
    for (const item of iterable) {
      Promise.resolve(item).then(resolve, reject);
    }
  });
}
// Empty iterable → hangs forever (per spec)

```

5. How do Web Workers, Worker Threads, and SharedArrayBuffer work?

Answer: All move work off the main thread, but with different models and constraints.

Web Workers (browser):

- Separate thread with **no DOM access**
- Communication via `postMessage` (structured clone) or `Transferable` objects (zero-copy move)
- Ideal for: parsing, crypto, image processing, heavy computation

```
// main.js
const worker = new Worker("worker.js");
worker.postMessage({ data: largeArrayBuffer },
[largeArrayBuffer]); // transfer ownership

worker.onmessage = (e) => updateUI(e.data);

// worker.js
self.onmessage = (e) => {
  const result = heavyCompute(e.data);
  self.postMessage(result);
};
```

Worker Threads (Node.js):

- `worker_threads` module — true threads sharing process memory options
- `SharedArrayBuffer` + `Atomics` for lock-free shared memory between threads

```
import { Worker, isMainThread, workerData, parentPort }
from "node:worker_threads";

if (isMainThread) {
  const worker = new Worker(new URL(import.meta.url));
  worker.on("message", console.log);
  worker.postMessage({ n: 40 });
} else {
  const fib = (n) => (n <= 1 ? n : fib(n - 1) + fib(n - 2));
  parentPort.postMessage(fib(workerData?.n ?? 40));
}
```

SharedArrayBuffer:

- Raw binary buffer shared between threads
- Requires **cross-origin isolation** headers in browsers (`Cross-Origin-Opener-Policy`, `Cross-Origin-Embedder-Policy`) due to Spectre mitigations
- Use `Atomics.wait` / `Atomics.notify` for synchronization

When to choose what:

Tool	Environment	Shared memory	DOM
Web Worker	Browser	No (message passing)	No
Service Worker	Browser	No	No
Worker Threads	Node.js	Optional (SAB)	N/A

6. Design and implement an EventEmitter / pub-sub system

Answer: A foundational pattern in Node.js, browser event targets, and frontend state architectures.

```
class EventEmitter {
  #events = new Map();

  on(event, listener) {
    if (!this.#events.has(event))
      this.#events.set(event, new Set());
    this.#events.get(event).add(listener);
    return () => this.off(event, listener); //
  }
  unsubscribe handle
```

```

}

once(event, listener) {
  const wrapper = (...args) => {
    this.off(event, wrapper);
    listener(...args);
  };
  wrapper.original = listener;
  return this.on(event, wrapper);
}

off(event, listener) {
  const set = this.#events.get(event);
  if (!set) return this;
  for (const fn of set) {
    if (fn === listener || fn.original === listener)
set.delete(fn);
  }
  return this;
}

emit(event, ...args) {
  const set = this.#events.get(event);
  if (!set) return false;
  for (const listener of [...set]) { // copy - safe
if listener mutates during emit
    listener(...args);
  }
  return true;
}
}

```

Production concerns:

- **Memory leaks** — always remove listeners; use `AbortSignal` pattern for auto-cleanup
- **Error handling** — Node.js `EventEmitter` emits `error` events specially; unhandled `error` crashes the process
- **Max listeners warning** — signal of leak
(`emitter.setMaxListeners`)
- **Async listeners** — `emit` does not await; use `emitAsync` pattern if needed

Pub/sub vs EventEmitter:

- **EventEmitter** — in-process, object-bound, synchronous by default
- **Pub/sub (message broker)** — decoupled publishers/subscribers, often cross-process (Redis, NATS, MQTT)

7. Explain property descriptors, Symbols, and metaprogramming

Answer: Beyond `Proxy`, senior developers use low-level object APIs for libraries, frameworks, and defensive code.

Property descriptors:

```
const obj = {};

Object.defineProperty(obj, "id", {
  value: 42,
  writable: false,          // cannot change value
  enumerable: false,        // hidden from for...in /
Object.keys
  configurable: false,      // cannot delete or reconfigure
});

Object.defineProperties(obj, {
  name: { value: "Alex", writable: true, enumerable:
true },
  secret: { get() { return "***"; }, enumerable: false
},
});
```

Symbols — unique, non-enumerable keys:

```
const ITERATE = Symbol.iterator;
const META = Symbol("meta");

const collection = {
  items: [1, 2, 3],
  [Symbol.iterator]() {
    let i = 0;
    const items = this.items;
    return {
      next: () => ({ value: items[i], done: i++ >=
```

```

items.length })),
    };
},
};

// Well-known symbols: Symbol.toStringTag,
Symbol.toPrimitive, Symbol.hasInstance

```

Metaprogramming use cases:

API	Use case
<code>Object.defineProperty</code>	Framework internals, lazy getters
<code>Symbol</code>	Avoid naming collisions, customize built-in behavior
<code>Proxy</code>	Reactivity (Vue), validation, logging
<code>Reflect</code>	Default behavior inside Proxy traps

Interview trap: `for...in` skips non-enumerable properties; `Object.keys` skips Symbols. Use `Reflect.ownKeys()` for everything.

8. How does V8 optimize JavaScript execution?

Answer: V8 compiles JavaScript through multiple tiers — no longer purely "interpreted vs compiled."

Pipeline (simplified):

```
Source → Parser → Ignition (bytecode) → Sparkplug  
(baseline JIT)  
  
→ Maglev (mid-  
tier JIT)  
  
→ TurboFan  
(optimizing JIT)
```

Hidden classes (Maps):

- V8 assigns internal "shapes" to objects based on property addition order
- Objects with same shape share hidden class → faster property access

```
// BAD — different shapes (deopt risk)  
function Point(x, y) {  
  this.x = x;  
  this.y = y;  
}  
  
const a = new Point(1, 2);  
const b = new Point(1, 2);  
b.z = 3; // different hidden class than a  
  
// GOOD — consistent initialization order
```

```
function Point(x, y, z = null) {  
  this.x = x;  
  this.y = y;  
  this.z = z;  
}
```

Deoptimization triggers:

- Adding properties to objects after creation in inconsistent order
- Changing property types (int → string)
- Arguments object usage in hot functions
- `try/catch` in tight loops (less problematic in modern V8)
- `delete obj.prop` — changes hidden class

Senior advice:

- Write **monomorphic** code in hot paths (same types, same object shapes)
 - Avoid micro-optimizing prematurely — profile first with `--prof` or Chrome DevTools
 - `JSON.parse` + typed arrays often beat hand-parsed loops for bulk data
-

9. What are backpressure and streaming patterns in Node.js?

Answer: Backpressure occurs when a data producer is faster than the consumer. Without handling it, memory buffers grow unbounded.

Node.js streams:

```
import { createReadStream, createWriteStream } from
"node:fs";
import { pipeline } from "node:stream/promises";
import { createGzip } from "node:zlib";

// pipeline handles backpressure and errors
automatically
await pipeline(
  createReadStream("input.csv"),
  createGzip(),
  createWriteStream("output.csv.gz")
);
```

How backpressure works:

```
Readable → push data → Writable buffer full?
└─ yes → pause readable
(highWaterMark)
└─ no → continue
Writable drains → emit 'drain' → resume readable
```

Manual backpressure:

```
function writeWithBackpressure(writable, chunks) {
  let i = 0;

  function writeNext() {
    let ok = true;
    while (i < chunks.length && ok) {
      ok = writable.write(chunks[i++]);
    }
    if (i < chunks.length) {
      writable.once("drain", writeNext);
    } else {
      writable.end();
    }
  }

  writeNext();
}
```

2026 patterns:

- `pipeline()` over chaining `.pipe()` — proper error propagation and cleanup
- Async iterables for streaming transforms
- Web Streams API converging with Node.js streams
(`ReadableStream` interop in Node 18+)

10. Implement function composition — `compose` and

`pipe`

Answer: Composition builds complex operations from small pure functions — core to functional architecture and middleware patterns (Redux, Express).

```
const pipe = (...fns) => (input) =>
  fns.reduce((acc, fn) => fn(acc), input);

const compose = (...fns) => (input) =>
  fns.reduceRight((acc, fn) => fn(acc), input);

const trim = (s) => s.trim();
const toLower = (s) => s.toLowerCase();
const removeSpaces = (s) => s.replace(/\s+/g, "-");

const slugify = pipe(trim, toLower, removeSpaces);
slugify("  Hello World  "); // "hello-world"

// compose is pipe in reverse order
const slugify2 = compose(removeSpaces, toLower, trim);
```

With async functions:

```
const pipeAsync = (...fns) => async (input) => {
  let result = input;
  for (const fn of fns) {
    result = await fn(result);
  }
}
```

```
    }  
    return result;  
};  
  
const fetchUser = (id) => () =>  
  fetch(`/api/users/${id}`).then(r => r.json());  
const extractName = (user) => user.name;  
const greet = (name) => `Hello, ${name}`;  
  
// Middleware-style (Express/Koa pattern)  
const middleware = [  
  async (ctx, next) => { ctx.start = Date.now(); await  
    next(); },  
  async (ctx, next) => { await next(); ctx.duration =  
    Date.now() - ctx.start; },  
];
```

Senior insight: Composition requires **consistent return types** between steps. Document and enforce contracts — TypeScript makes this tractable at scale.

11. How do you architect JavaScript for microfrontends?

Answer: Microfrontends split a UI into independently deployable apps. JavaScript architecture must solve **runtime integration**, **shared dependencies**, and **isolation**.

Integration approaches:

Approach	How	Trade-off
Build-time	npm packages, monorepo	Simple, no runtime isolation
Module Federation	Webpack/Rspack runtime sharing	Shared deps, lazy load remotes
Web Components	Custom elements as boundaries	Framework-agnostic, shadow DOM
iframe	Full isolation	Heavy, poor UX integration
Single-SPA	Route-based orchestration	Meta-framework overhead

Module Federation (conceptual):

```
// Host webpack.config
new ModuleFederationPlugin({
  name: "shell",
  remotes: {
    checkout:
      "checkout@https://cdn.example.com/checkout/remoteEntry."
```

```
js",
  },
  shared: {
    react: { singleton: true, requiredVersion:
"^19.0.0" },
    "react-dom": { singleton: true },
  },
});

// Host runtime
const CheckoutApp = React.lazy(() =>
import("checkout/App"));
```

Critical decisions:

1. **Shared dependency strategy** — one React instance (singleton) vs duplicated bundles
2. **CSS isolation** — shadow DOM, CSS modules, BEM prefixes
3. **Routing** — who owns the URL? shell vs remotes
4. **State sharing** — custom events, shared store, URL state, backend as source of truth
5. **Versioning** — semver contracts between teams; canary remote deployments

Anti-patterns: Two Reacts on one page, global CSS collisions, circular remote dependencies, shared mutable global state.

12. What are SSR and hydration pitfalls in isomorphic JavaScript?

Answer: **SSR** renders HTML on the server; **hydration** attaches client-side JavaScript event handlers and state to that HTML.

Common pitfalls:

Pitfall	Cause	Fix
Hydration mismatch	Server HTML $\neq$ client render	Avoid <code>Date.now()</code> , <code>Math.random()</code> , <code>window</code> in initial render
Double data fetch	Server fetched, client re-fetches	Pass serialized state (<code>__NEXT_DATA__</code> , <code>TransferState</code>)
Layout shift on hydrate	Client-only content appears	SSR same skeleton; defer client-only to <code>useEffect</code>
Memory pressure	Full app state on server per request	Stream HTML; limit per-request cache

Pitfall	Cause	Fix
Bundle bloat	Server code in client bundle	Separate entry points; <code>import 'server-only'</code>

```
// BAD - hydration mismatch
function Timestamp() {
  return <span>{new Date().toLocaleString()}</span>;
}

// GOOD - render placeholder on server, update on client
function Timestamp() {
  const [time, setTime] = useState(null);
  useEffect(() => setTime(new Date().toLocaleString()),
    []);
  return <span suppressHydrationWarning>{time ?? "..."}
</span>;
}
```

Isomorphic module pattern:

```
// api.js
export async function getUser(id, { fetchImpl = fetch }
= {}) {
  const res = await fetchImpl(`/api/users/${id}`);
  return res.json();
}
```

```
// server.js - inject server fetch with cookies
getUser(id, { fetchImpl: serverFetch });

// client.js - use browser fetch
getUser(id);
```

Senior talking point: Partial hydration and islands architecture (Astro, Qwik) reduce JS shipped to client — understand when full SPA hydration is overkill.

13. How do Service Workers enable offline-first applications?

Answer: Service Workers are event-driven scripts that act as a programmable network proxy between the app and the network.

Lifecycle:

```
Register → Install (cache static assets) → Activate
(cleanup old caches) → Fetch (intercept requests)
```

```
// sw.js
const CACHE = "app-v2";
const PRECACHE = ["/", "/index.html", "/app.js",
"/app.css"];

self.addEventListener("install", (event) => {
```

```

    event.waitUntil(caches.open(CACHE).then((c) =>
c.addAll(PRECACHE)));
    self.skipWaiting();
  });

self.addEventListener("activate", (event) => {
  event.waitUntil(
    caches.keys().then((keys) =>
      Promise.all(keys.filter((k) => k !==
CACHE).map((k) => caches.delete(k)))
    )
  );
  self.clients.claim();
});

self.addEventListener("fetch", (event) => {
  event.respondWith(
    caches.match(event.request).then((cached) => {
      return cached ||
fetch(event.request).then((response) => {
        // Optional: cache successful GET responses
(stale-while-revalidate)
        return response;
      });
    })
  );
});

```

Caching strategies:

Strategy	Pattern	Use case
Cache-first	Cache → network fallback	Static assets
Network-first	Network → cache fallback	API data
Stale-while-revalidate	Return cache, update in background	Semi-fresh content
Network-only	Always network	Auth, mutations

2026 considerations:

- Service Workers require HTTPS (localhost exempt)
- Coordinate cache busting with build hashes
- Background Sync and Push API for deferred actions
- Do not cache opaque responses blindly — CORS limitations

14. How do you structure and scale a large JavaScript codebase?

Answer: Senior developers own **boundaries**, **conventions**, and **enforcement** — not just folder layout.

Recommended structure (monorepo example):

```
apps/  
  web/           ← deployable application  
  admin/  
packages/  
  ui/           ← shared components  
  utils/        ← pure functions  
  api-client/   ← typed API layer  
  config-eslint/ ← shared tooling
```

Principles:

1. Feature-based modules over type-based

(`/features/checkout/` not `/components/` + `/services/` scattered)

2. Explicit public APIs — `index.ts` barrel exports only what is public

3. Dependency direction — apps → packages, never packages → apps

4. Lint boundaries — ESLint `import/no-restricted-paths`, Nx module boundary rules

5. Shared nothing by default — share deliberately with semver and changelogs

Code quality enforcement:

```
// eslint.config.js (flat config, 2026 standard)
export default [
  { rules: { "no-restricted-imports": ["error", {
    patterns: ["**/internal/*"] }} } },
];
```

Documentation: ADRs (Architecture Decision Records) for major choices — state management, routing, testing strategy.

Senior metrics: Build time, bundle size budgets, test coverage on critical paths, dependency update cadence, mean time to onboard a new developer.

15. What is JavaScript supply chain security and how do you harden it?

Answer: npm ecosystems are high-value attack targets. Senior engineers implement **defense in depth** beyond `npm audit`.

Threat vectors:

- **Dependency confusion** — typosquatting package names
- **Compromised maintainer accounts** — malicious patch releases

- **Install scripts** — `preinstall` / `postinstall` running arbitrary code
- **Prototype pollution** — vulnerable transitive dependencies
- **Lockfile tampering** — inconsistent installs across environments

Hardening checklist:

```
// package.json
{
  "scripts": {
    "preinstall": "npx only-allow pnpm"
  },
  "engines": {
    "node": ">=20.0.0"
  }
}
```

Control	Implementation
Lockfiles	Commit <code>pnpm-lock.yaml</code> / <code>package-lock.json</code> — CI uses <code>npm ci</code>
Minimal deps	Audit every new dependency; prefer stdlib / small libs

Control	Implementation
Pin versions	Renovate/Dependabot with review, not auto-merge to prod
Integrity	<code>npm audit</code> , Socket.dev, Snyk in CI
No install scripts	<code>"ignore-scripts": true</code> where safe; review exceptions
SBOM	Generate Software Bill of Materials for compliance
Private registry	Verdaccio / Artifactory for internal packages

Runtime defenses:

- CSP headers (see middle-level doc)
 - Subresource Integrity (SRI) for CDN scripts
 - `Object.freeze(Object.prototype)` in high-security contexts
 - Environment isolation — never run untrusted JS in same process as secrets
-

16. Implement deep equality with edge case handling

Answer: Production-grade deep equality must handle cycles, special types, and asymmetric structures.

```
function deepEqual(a, b, seen = new WeakMap()) {
  // Same reference or primitive equal
  if (Object.is(a, b)) return true;

  // NaN === NaN via Object.is above

  if (typeof a !== "object" || a === null ||
      typeof b !== "object" || b === null) {
    return false;
  }

  // Cycle detection
  if (seen.has(a)) return seen.get(a) === b;
  seen.set(a, b);

  // Date
  if (a instanceof Date && b instanceof Date) {
    return a.getTime() === b.getTime();
  }

  // RegExp
  if (a instanceof RegExp && b instanceof RegExp) {
    return a.source === b.source && a.flags ===
b.flags;
  }
}
```

```

}

// Map
if (a instanceof Map && b instanceof Map) {
  if (a.size !== b.size) return false;
  for (const [key, val] of a) {
    if (!b.has(key) || !deepEqual(val, b.get(key),
seen)) return false;
  }
  return true;
}

// Set
if (a instanceof Set && b instanceof Set) {
  if (a.size !== b.size) return false;
  for (const val of a) {
    if (![...b].some((bVal) => deepEqual(val, bVal,
seen))) return false;
  }
  return true;
}

// TypedArray
if (ArrayBuffer.isView(a) && ArrayBuffer.isView(b)) {
  if (a.byteLength !== b.byteLength) return false;
  return a.every((v, i) => v === b[i]);
}

// Plain objects / arrays
const keysA = Reflect.ownKeys(a);
const keysB = Reflect.ownKeys(b);
if (keysA.length !== keysB.length) return false;

```

```
return keysA.every((key) =>
  keysB.includes(key) && deepEqual(a[key], b[key],
seen)
);
}
```

Edge cases to discuss:

- `+0` vs `-0` — `Object.is` distinguishes; `===` does not
- `NaN` — only equal to itself via `Object.is`
- Prototype chain — typically compare own properties only (as above)
- `structuredClone` for copying; `deepEqual` for comparison — different problems

17. Compare Promises, Observables, and async iterators

Answer: Three async abstractions with different **** cardinality**** and **cancellation** models.

Feature	Promise	Async Iterator	Observable
Values	Single	Multiple (pull)	Multiple (push)
Lazy	No (eager on creation)	Yes	Yes
Cancellation	AbortSignal (manual)	<code>break</code> / <code>return()</code>	Unsubscribe
Standard	Native	Native (ES2018+)	RxJS (de facto)

```
// Promise – one shot
const user = await fetchUser(id);

// Async iterator – pull stream
async function* paginate(url) {
  let next = url;
  while (next) {
    const page = await fetch(next).then(r => r.json());
    yield page.items;
    next = page.nextPage;
  }
}
```

```
for await (const items of paginate("/api/items")) {
  render(items);
}

// Observable (RxJS) — push stream with operators
import { fromEvent } from "rxjs";
import { debounceTime, map } from "rxjs/operators";

fromEvent(input, "input")
  .pipe(debounceTime(300), map(e => e.target.value))
  .subscribe(query => search(query));
// unsubscribe() cancels
```

When to choose:

- **Promise** — single async result (API call, file read)
- **Async iterator** — paginated APIs, file line reading, GraphQL @defer streams
- **Observable** — user events, WebSockets, complex stream composition with operators

2026 note: `ReadableStream` async iteration is increasingly used for fetch response bodies and SSE. RxJS remains common in Angular codebases.

18. How do you diagnose performance in long-running SPAs?

Answer: SPAs that stay open for hours face unique issues: memory growth, stale closures, degraded rendering, and state bloat.

Diagnostic workflow:

1. Reproduce under real session length (not fresh page load)
2. Chrome Performance → record during slow interaction (INP)
3. Memory → heap snapshot diff after 10/30/60 min
4. Coverage → find dead code in bundle
5. Lighthouse CI → regression gates in PR

Common SPA-specific issues:

Symptom	Likely cause	Fix
Tab slows after 30 min	Memory leak (listeners, caches)	Audit teardown, WeakMap caches
Scroll jank	Layout thrashing, large lists	Virtualization, <code>content-visibility</code>

Symptom	Likely cause	Fix
Slow route changes	Large component trees re-mounting	Code splitting, keep-alive patterns
Growing Redux/Zustand store	Unbounded history	Pagination, state pruning, TTL
Main thread blocked	Sync JSON parse of large payload	Web Worker, streaming parse

```
// Long task observer – detect blocking work > 50ms
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.warn("Long task:", entry.duration, "ms");
  }
});
observer.observe({ type: "longtask", buffered: true });
```

Performance budgets (senior ownership):

```
// bundlewatch or size-limit in CI
{ "path": "dist/main-*.js", "limit": "250 KB" }
```

2026 Core Web Vitals focus: INP (Interaction to Next Paint) replaced FID — optimize event handler duration and rendering

after interaction.

19. When and how do you integrate WebAssembly with JavaScript?

Answer: WebAssembly (Wasm) runs near-native code in a sandboxed VM. Use it when JavaScript is the bottleneck — not as a default.

Good use cases:

- Image/video codecs, cryptography, compression
- Game physics, audio DSP
- Parsing large files (CSV, protobuf)
- Porting existing C/C++/Rust libraries

Browser integration:

```
// Load and instantiate Wasm module
const response = await fetch("/add.wasm");
const { instance } = await
WebAssembly.instantiateStreaming(response, {
  env: {
    consoleLog: (n) => console.log(n), // imported JS
function
  },
```

```
});

instance.exports.add(2, 3); // call exported Wasm
function

// Share memory between JS and Wasm
const memory = new WebAssembly.Memory({ initial: 1 });
// 64KB pages
const wasmModule = await
WebAssembly.instantiateStreaming(fetch("/proc.wasm"), {
  env: { memory },
});
const view = new Uint8Array(memory.buffer);
```

Component Model (2026 direction):

- **Wasm Component Model** — compose modules across languages
- **JavaScript Promise Integration** — Wasm async calls integrate with JS Promises
- **Tooling:** Rust + `wasm-pack`, AssemblyScript for JS-like syntax

Trade-offs:

Pros	Cons
Near-native speed	No direct DOM access

Pros	Cons
Sandboxed	Boundary crossing has overhead
Portable binary	Debugging harder than JS
Reuse native libs	Bundle size for small functions

Rule of thumb: Profile first. If JS spends 90% of time in one hot loop on large data — Wasm candidate. If I/O bound — optimize network and caching instead.

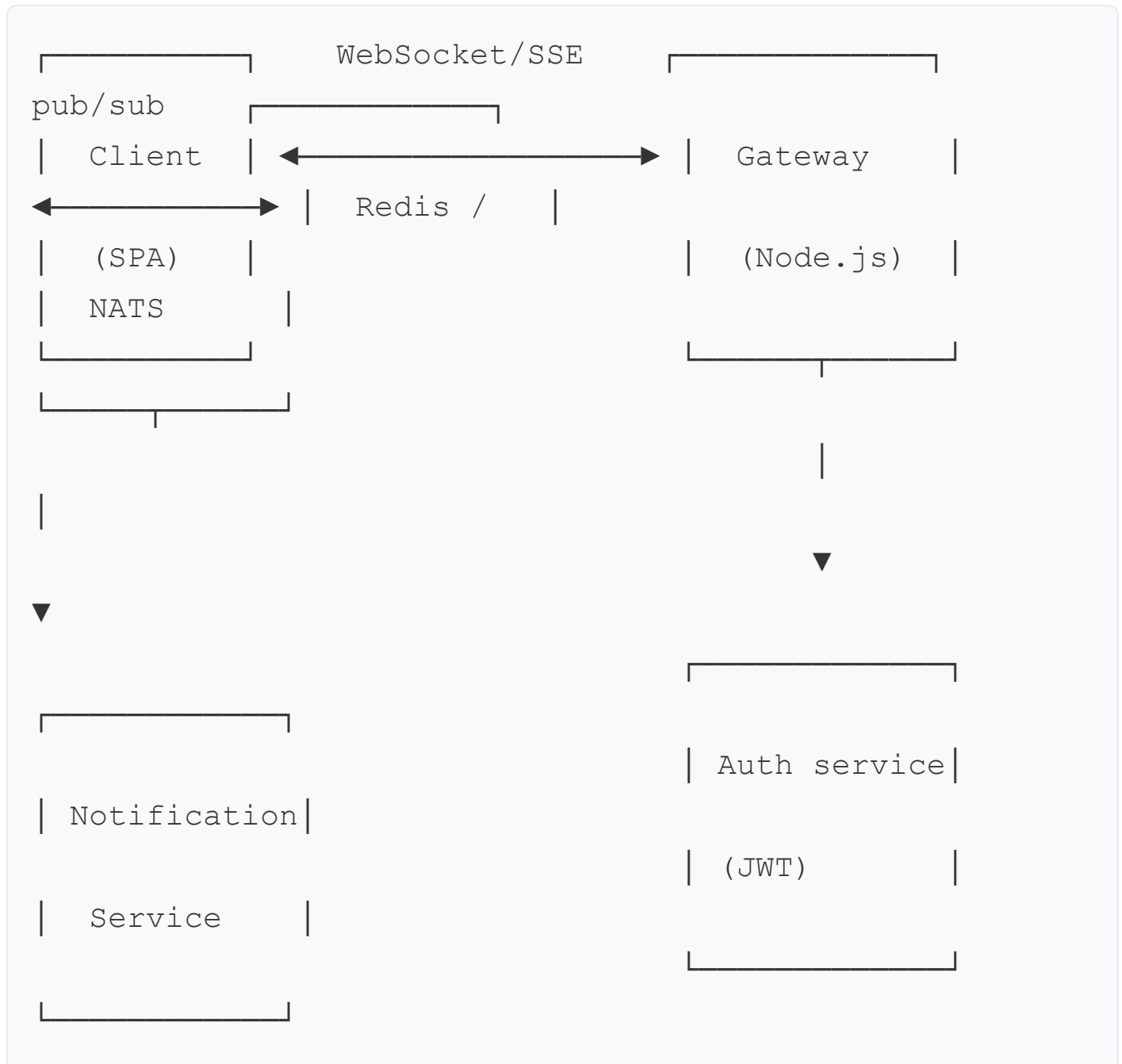
20. System design: build a real-time notification system in JavaScript

Answer: A classic senior system design question testing architecture, scaling, and JavaScript-specific constraints.

Requirements (typical):

- Push notifications to millions of connected clients
- At-least-once delivery acceptable; exactly-once for critical alerts
- Support web (browser) and mobile
- Handle reconnects gracefully

High-level architecture:



Client-side (JavaScript):

```

class NotificationClient {
  #ws = null;
  #retryDelay = 1000;
  #maxDelay = 30000;
  #listeners = new Set();
}
  
```

```

connect(token) {
  this.#ws = new WebSocket(`wss://api.example.com/ws?
token=${token}`);

  this.#ws.onmessage = (e) => {
    const msg = JSON.parse(e.data);
    this.#listeners.forEach((fn) => fn(msg));
  };

  this.#ws.onclose = () => this.#reconnect(token);
  this.#ws.onerror = () => this.#ws.close();
}

#reconnect(token) {
  setTimeout(() => {
    this.#retryDelay = Math.min(this.#retryDelay * 2,
this.#maxDelay);
    this.connect(token);
  }, this.#retryDelay);
}

subscribe(fn) {
  this.#listeners.add(fn);
  return () => this.#listeners.delete(fn);
}
}

```

Key design decisions to articulate:

Decision	Options	Recommendation
Transport	WebSocket vs SSE vs long-poll	WebSocket bidirectional; SSE simpler for server→client only
Scale gateways	Sticky sessions vs shared state	Sticky + Redis pub/sub for cross-node broadcast
Missed messages	Last-event-id / cursor	Client sends <code>lastSeenId</code> on reconnect; server replays from queue
Auth	Token in query vs first message	Short-lived JWT; refresh before expiry
Backpressure	Drop vs queue vs slow consumer disconnect	Rate limit per client; priority queues for critical alerts
Heartbeat	Ping/pong every 30s	Detect dead connections; load balancer timeout alignment

Failure modes:

- Thundering herd on reconnect → **jittered exponential backoff**
- Gateway crash → client reconnects to another instance via load balancer
- Message duplication → idempotent handlers with `messageId` dedup on client

Senior closing: Start simple (single Node.js gateway + Redis), measure connection count and memory per socket, then shard gateways horizontally. Premature Kafka is an anti-pattern until proven necessary.

Quick Study Tips

1. **Whiteboard a Promise** — full `then` chain with assimilation before any interview.
2. **Explain one system end-to-end** — notifications, autocomplete, or file upload with progress.
3. **Profile real code** — bring a story about finding and fixing a memory leak or long task.
4. **Know your trade-offs** — not just "use X" but when X is wrong.
5. **Stay current** — ECMAScript proposals, Node.js LTS schedule, bundler landscape (Vite, Rspack, Turbopack).